SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING
(SAVEETHASCHOOLOFENGINEERING)

Department of Computer Science & Engineering

CO2 - ASSESSMENT TOOL -1

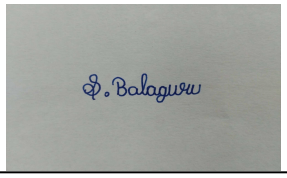
Scenario-Based Assessment

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO2**	**Assessment**	**Assessment Tool 1: Scenario-Based Assessment**
Weight age	**50 %**	**Total Marks**	**100**
CO2Statement	***Design effective test cases using black-box, white-box testing techniques and to control methods flow***		
Student Name	**Balaguru S**	**Reg.No.**	**192312070**
Section/Batch	**Slot A / 2023**	**Date**	**30-10-2026**

Code of Conduct:

I, Balaguru S (192312070), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment .I understands that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



3. A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.

Black-Box Test Design for a Retail E-Commerce Order Module (Price, Quantity, Discount).

1. Analyse the scenario and identify the inputs, business rules and failure context.

2. Evaluate the limitations of the current black-box testing approach.

3. Apply equivalence class partitioning to price, quantity and discount.

4. Apply boundary value analysis, including the internal tier boundaries.

5. Apply decision table testing and other black-box methods (interaction matrix, error guessing).

6. Propose a revised test design strategy that covers all input combinations.

7. Justify the approach in terms of defect detection and test completeness.

1. Problem Understanding and Context Analysis

1.1 Scenario summary

A retail e-commerce system calculates the payable amount of an order line from three values typed in by the user or supplied by an upstream page: **price**, **quantity** and **discount**. The test team applied black-box testing, and every field passed its own validation checks, yet defects still reached users. The escapes were reported only for **specific input combinations**, for example a particular quantity used together with a particular discount.

This symptom is the key to the whole analysis. A defect that appears only when two or more fields take certain values together is not a single-field validation defect. It is an **interaction defect**, and it points to a conditional business rule in which one field changes what is allowed for another. A test design that examines each field separately cannot, by construction, expose such a rule.

1.2 Business rules assumed for the analysis

The question does not supply a specification, so a representative one is assumed. It is typical of retail order entry and is used only to make the reasoning and the test data concrete. All amounts are in Indian rupees (Rs.).

ID	Business rule (assumed specification)
BR1	**Price:** decimal value from Rs. 1.00 to Rs. 10,000.00 with at most two decimal places.
BR2	**Quantity:** whole number from 1 to 100.
BR3	**Discount:** whole percentage from 0 to 50.
BR4	**Discount cap depends on quantity:** quantity 1 to 9 allows at most 10 %; quantity 10 to 49 allows at most 25 %; quantity 50 to 100 allows at most 50 %.
BR5	**Line total** = Price x Quantity x (1 - Discount / 100), rounded half-up to two decimals.
BR6	Any invalid input rejects the order. The system must report every faulty field in one response, not only the first.

Rule BR4 is the source of the interaction. Quantity and discount are each valid on their own range, but a particular discount is legal only for certain quantities. That single rule turns the 100 x 51 grid of quantity and discount values into twelve distinct behaviours (three quantity tiers by four discount bands), as Section 3.4 shows.

1.3 Size of the input space

Exhaustive testing of the valid domain alone would need 999,901 prices x 100 quantities x 51 discounts = **5,099,495,100** test cases. At one test per second this takes about 162 years, and it still ignores invalid and badly formatted input. "Covering all possible input combinations" therefore cannot mean testing every value. It must mean covering every **distinct combination of behaviour**: every combination of equivalence classes, every boundary, and every rule of the decision logic. The rest of this answer builds a suite that does exactly that.

2. Analysis of the Current Approach and Its Limitations

2.1 Current approach inferred from the failure pattern

No test documentation is given, so the current approach is inferred from what it missed. The evidence (field-level checks pass, combination checks fail) is consistent with testers who applied equivalence partitioning and boundary value analysis **field by field**, holding the other two fields at fixed, comfortable valid values (for example price 500, quantity 5, discount 5 %). This is a common and reasonable-looking practice, and it is the baseline assumed for the comparisons in Section 5.

2.2 Limitations of the current approach

#	Limitation	Why it hides defects	Example of an escaped defect
L1	Fields tested in isolation	Each test varies one field while the others stay at fixed nominal values, so the tests never leave a single quantity/discount combination.	Quantity 5 with discount 15 % is accepted although BR4 allows only 10 %.
L2	Partitions built from field ranges only	Plain ECP sees quantity 1-100 and discount 0-50 as one class each. BR4 splits both into sub-classes (3 tiers, 4 bands) that are never combined.	Twelve behaviours are tested as one or two.
L3	Boundaries only at the ends of ranges	The internal boundaries (quantity 9/10 and 49/50, caps 10/25/50) are not the end of any field range, so BVA never visits them.	Tier switch coded as quantity > 10 instead of >= 10.
L4	No decision table or rule traceability	The multi-condition logic has no explicit rule list, so untested rules are invisible to reviewers.	Two faulty fields reported as one error message.
L5	Round, convenient test data	Values such as 100.00 x 10 x 10 % never produce a half-cent, so precision defects stay hidden.	169.915 stored as 169.91 by floating-point arithmetic.
L6	Happy-path and single-fault bias	Invalid and badly formatted input is tried rarely and almost never in combination.	Non-numeric discount produces NaN instead of an error.

2.3 Worked example of an escaped defect

Suppose the developer codes the tier switch as 'quantity > 10' where BR4 requires 'quantity >= 10'. Quantity 10 is then wrongly given the 10 % cap. Every per-field test still passes, because each field's value is legal in isolation. Only the combination exposes the fault.

Test	Quantity	Discount	Result under the defect
Per-field quantity test (discount 5 %)	10	5	Pass: 5 % is within both caps
Per-field discount test (quantity 100)	100	11	Pass: cap is 50 % at quantity 100
Combination test TC-C03	**10**	**11**	**Fail: wrongly rejected (cap treated as 10 %, spec says 25 %)**

2.4 Evaluation

The current approach is adequate for single-field validation: in the seeded-defect analysis of Section 5 it detects 6 of 11 defects, all of them field-level. It is, however, structurally blind to interaction defects, internal boundaries, multi-fault handling and computation precision. Adding more tests of the same kind would not help, because the missing coverage is of a different kind. What is needed is a different set of techniques aimed at combinations, which the next section applies.

3. Application of Black-Box Techniques

3.1 Equivalence class partitioning (ECP)

Each input domain is partitioned into classes whose members are expected to behave identically. Valid classes are refined using BR4, which is the step the current approach missed: quantity splits into three tiers and discount into four bands. This gives 5 price classes, 7 quantity classes and 7 discount classes, **19 classes in total**.

Field	Class	Description	Type	Representative value
Price	P-V1	1.00 to 10,000.00, at most 2 decimals	Valid	200.00
Price	P-I1	Below 1.00 (zero or negative)	Invalid	-50.00
Price	P-I2	Above 10,000.00	Invalid	15,000.00
Price	P-I3	Non-numeric or blank	Invalid	abc
Price	P-I4	More than 2 decimal places	Invalid	150.505
Quantity	Q-V1	Tier 1: 1 to 9	Valid	5
Quantity	Q-V2	Tier 2: 10 to 49	Valid	20

Field	Class	Description	Type	Representative value
Quantity	Q-V3	Tier 3: 50 to 100	Valid	60
Quantity	Q-I1	Below 1 (zero or negative)	Invalid	-3
Quantity	Q-I2	Above 100	Invalid	150
Quantity	Q-I3	Not a whole number	Invalid	2.5
Quantity	Q-I4	Non-numeric or blank	Invalid	ten
Discount	D-V0	0 % (no discount)	Valid	0
Discount	D-V1	Band 1: 1 to 10 %	Valid	5
Discount	D-V2	Band 2: 11 to 25 %	Valid	15
Discount	D-V3	Band 3: 26 to 50 %	Valid	40
Discount	D-I1	Below 0	Invalid	-5
Discount	D-I2	Above 50	Invalid	75
Discount	D-I3	Non-numeric or blank	Invalid	ten

Two combination rules are applied. **Invalid classes are tested one at a time**, with every other field valid (the single-fault rule), so that one error cannot mask another. **Valid classes are combined systematically** in the interaction matrix of Section 3.4. The invalid-class tests use price 200.00, quantity 5, discount 5 % as the valid backdrop.

Test ID	Price	Qty	Disc %	Class targeted	Expected result
TC-E01	-50.00	5	5	Price < 1.00 (below range)	Reject: Price error
TC-E02	15,000.00	5	5	Price > 10,000 (above range)	Reject: Price error
TC-E03	abc	5	5	Price non-numeric	Reject: Price error
TC-E04	150.505	5	5	Price with > 2 decimals	Reject: Price error
TC-E05	200.00	-3	5	Quantity < 1	Reject: Quantity error
TC-E06	200.00	150	5	Quantity > 100	Reject: Quantity error
TC-E07	200.00	2.5	5	Quantity not an integer	Reject: Quantity error

Test ID	Price	Qty	Disc %	Class targeted	Expected result
TC-E08	200.00	ten	5	Quantity non-numeric	Reject: Quantity error
TC-E09	200.00	5	-5	Discount < 0	Reject: Discount range error
TC-E10	200.00	5	75	Discount > 50	Reject: Discount range error
TC-E11	200.00	5	ten	Discount non-numeric	Reject: Discount range error

3.2 Boundary value analysis (BVA)

Defects cluster at the edges of ranges, so each numeric field is tested at its minimum and maximum using the three-value form (just below, on, and just above each edge), with robustness values outside the range. For price the edges are 1.00 and 10,000.00, for quantity 1 and 100, for discount 0 and 50. Neutral values for the other two fields are chosen so that no rule other than the one under test can interfere: price tests use quantity 1 and discount 0; quantity tests use price 100.00 and discount 0; discount tests use price 100.00 and quantity 100, where the cap is 50 %.

Test ID	Field	Price	Qty	Disc %	Expected result
TC-B01	Price	0.99	1	0	Reject: Price error
TC-B02	Price	1.00	1	0	Accept; total = 1.00
TC-B03	Price	1.01	1	0	Accept; total = 1.01
TC-B04	Price	9999.99	1	0	Accept; total = 9,999.99
TC-B05	Price	10000.00	1	0	Accept; total = 10,000.00
TC-B06	Price	10000.01	1	0	Reject: Price error
TC-B07	Quantity	100.00	0	0	Reject: Quantity error
TC-B08	Quantity	100.00	1	0	Accept; total = 100.00
TC-B09	Quantity	100.00	2	0	Accept; total = 200.00

Test ID	Field	Price	Qty	Disc %	Expected result
TC-B10	Quantity	100.00	99	0	Accept; total = 9,900.00
TC-B11	Quantity	100.00	100	0	Accept; total = 10,000.00
TC-B12	Quantity	100.00	101	0	Reject: Quantity error
TC-B13	Discount	100.00	100	-1	Reject: Discount range error
TC-B14	Discount	100.00	100	0	Accept; total = 10,000.00
TC-B15	Discount	100.00	100	1	Accept; total = 9,900.00
TC-B16	Discount	100.00	100	49	Accept; total = 5,100.00
TC-B17	Discount	100.00	100	50	Accept; total = 5,000.00
TC-B18	Discount	100.00	100	51	Reject: Discount range error

These 18 tests cover the range edges. The **internal** boundaries created by BR4 are handled by the interaction boundary tests in Section 3.4.

3.3 Decision table testing

Decision table testing makes the rule logic explicit. Four conditions drive the outcome: **C1** price valid, **C2** quantity valid, **C3** discount in range 0 to 50, **C4** discount within the cap for the quantity (Y = yes, N = no, - = not applicable). C4 is meaningful only when C2 and C3 are both true, because the cap cannot be evaluated without a valid quantity and discount. The full table has 2^4 = 16 columns; applying the "don't care" for C4 collapses it to **10 rules**, and each rule gets one test.

Rule / Test	C1	C2	C3	C4	Price	Qty	Disc	Expected result
TC-D01	Y	Y	Y	Y	500.00	10	20	Accept; total = 4,000.00
TC-D02	Y	Y	Y	N	500.00	5	15	Reject: Discount exceeds cap
TC-D03	N	Y	Y	Y	0.50	10	20	Reject: Price error

Rule / Test	C1	C2	C3	C4	Price	Qty	Disc	Expected result
TC-D04	N	Y	Y	N	0.50	5	15	Reject: Price error + Discount exceeds cap
TC-D05	Y	N	Y	-	500.00	0	10	Reject: Qty error
TC-D06	Y	Y	N	-	500.00	10	60	Reject: Discount range error
TC-D07	Y	N	N	-	500.00	101	-5	Reject: Qty error + Discount range error
TC-D08	N	N	Y	-	-10.00	0	10	Reject: Price error + Qty error
TC-D09	N	Y	N	-	10001.00	10	abc	Reject: Price error + Discount range error
TC-D10	N	N	N	-	(blank)	-1	75	Reject: Price error + Qty error + Discount range error

The table also exposes the requirement in BR6 that all faults are reported together: rules R4 and R7 to R10 each need more than one message. A system that stops at the first error fails these tests, a defect the isolated field tests could never reveal.

3.4 Interaction testing: tier-by-band matrix and cap boundaries

Equivalence classes of different fields must be **combined**. Quantity has three valid tiers and discount four valid bands. Price has a single valid class, so combining all valid classes of all three fields gives 1 x 3 x 4 = **12 combinations**. (Because price has only one valid class, even all-pairs testing would reduce to these same twelve cells.) Each cell receives one test, with a representative value from each class and price 200.00. Figure 1 shows the matrix, with rejected cells marking where BR4 forbids the combination.

	D0 0 %	D1 1-10 %	D2 11-25 %	D3 26-50 %
T1 Qty 1-9	M01 Accept	M02 Accept	M03 Reject (cap)	M04 Reject (cap)
T2 Qty 10-49	M05 Accept	M06 Accept	M07 Accept	M08 Reject (cap)
T3 Qty 50-100	M09 Accept	M10 Accept	M11 Accept	M12 Accept

Figure 1: Tier-by-band interaction matrix (12 valid class combinations, TC-M01 to TC-M12)

Test ID	Tier	Band	Price	Qty	Disc %	Expected result
TC-M01	T1	D0	200.00	5	0	Accept; total = 1,000.00
TC-M02	T1	D1	200.00	5	5	Accept; total = 950.00
TC-M03	T1	D2	200.00	5	15	Reject: Discount exceeds 10% cap
TC-M04	T1	D3	200.00	5	40	Reject: Discount exceeds 10% cap
TC-M05	T2	D0	200.00	20	0	Accept; total = 4,000.00
TC-M06	T2	D1	200.00	20	5	Accept; total = 3,800.00
TC-M07	T2	D2	200.00	20	15	Accept; total = 3,400.00
TC-M08	T2	D3	200.00	20	40	Reject: Discount exceeds 25% cap
TC-M09	T3	D0	200.00	60	0	Accept; total = 12,000.00
TC-M10	T3	D1	200.00	60	5	Accept; total = 11,400.00
TC-M11	T3	D2	200.00	60	15	Accept; total = 10,200.00

Test ID	Tier	Band	Price	Qty	Disc %	Expected result
TC-M12	T3	D3	200.00	60	40	Accept; total = 7,200.00

Cell coverage is not enough, because defects also sit on the edges **between** cells. Interaction boundary tests probe each tier switch and cap from both sides: just inside and just outside the cap at quantities 9, 10, 49 and 50. Figure 2 plots these points against the cap staircase defined by BR4.

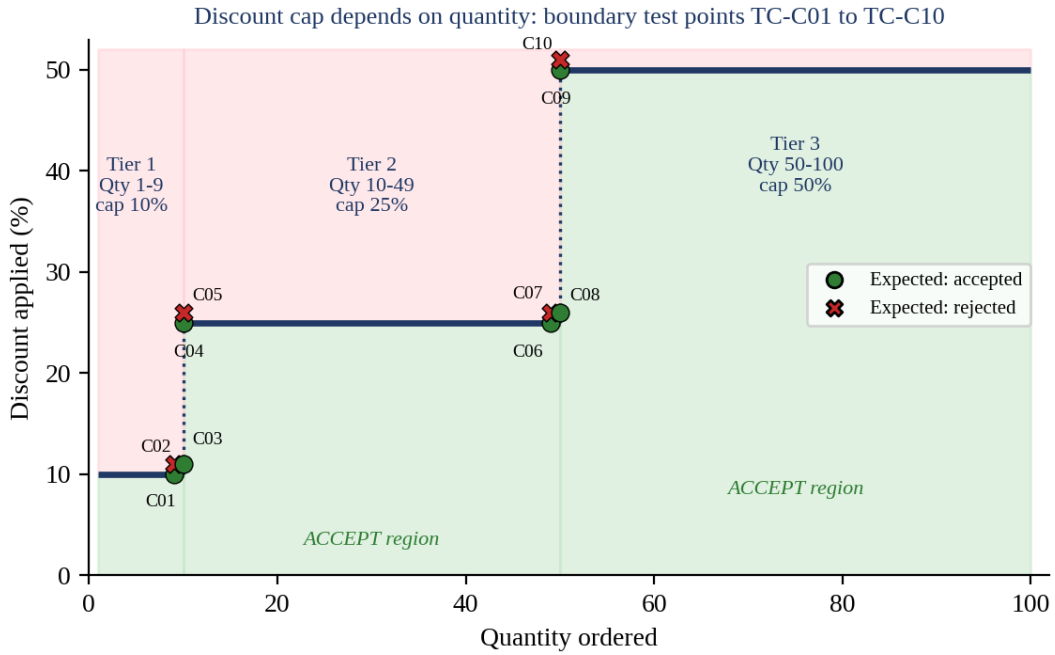


Figure 2: Discount cap versus quantity with interaction boundary tests TC-C01 to TC-C10

Test ID	Price	Qty	Disc %	Expected result
TC-C01	100.00	9	10	Accept; total = 810.00
TC-C02	100.00	9	11	Reject: Discount exceeds 10% cap
TC-C03	100.00	10	11	Accept; total = 890.00
TC-C04	100.00	10	25	Accept; total = 750.00
TC-C05	100.00	10	26	Reject: Discount exceeds 25% cap

Test ID	Price	Qty	Disc %	Expected result
TC-C06	100.00	49	25	Accept; total = 3,675.00
TC-C07	100.00	49	26	Reject: Discount exceeds 25% cap
TC-C08	100.00	50	26	Accept; total = 3,700.00
TC-C09	100.00	50	50	Accept; total = 2,500.00
TC-C10	100.00	50	51	Reject: Discount range error

3.5 Error guessing

Experience-based tests target faults that formal partitioning does not predict: precision, number formats, overflow and empty submission.

Test ID	Price	Qty	Disc %	Fault targeted	Expected result
TC-G01	19.99	10	15	Rounding: 199.90 x 0.85 = 169.915 must round half-up to 169.92	Accept; total = 169.92
TC-G02	1e3	5	5	Scientific notation in price	Reject: Price error (format)
TC-G03	200.00	2147483648	5	Integer-overflow quantity	Reject: Quantity error (no crash)
TC-G04	(blank)	(blank)	(blank)	All fields empty / submit twice	Reject: all three field errors; no duplicate order

4. Revised Test Design Strategy

4.1 Overall strategy

The revised strategy layers the techniques so that each layer targets a defect type the previous layer cannot see. Figure 3 shows the workflow and Table 4.1 lists the resulting suite of **65 test cases**.

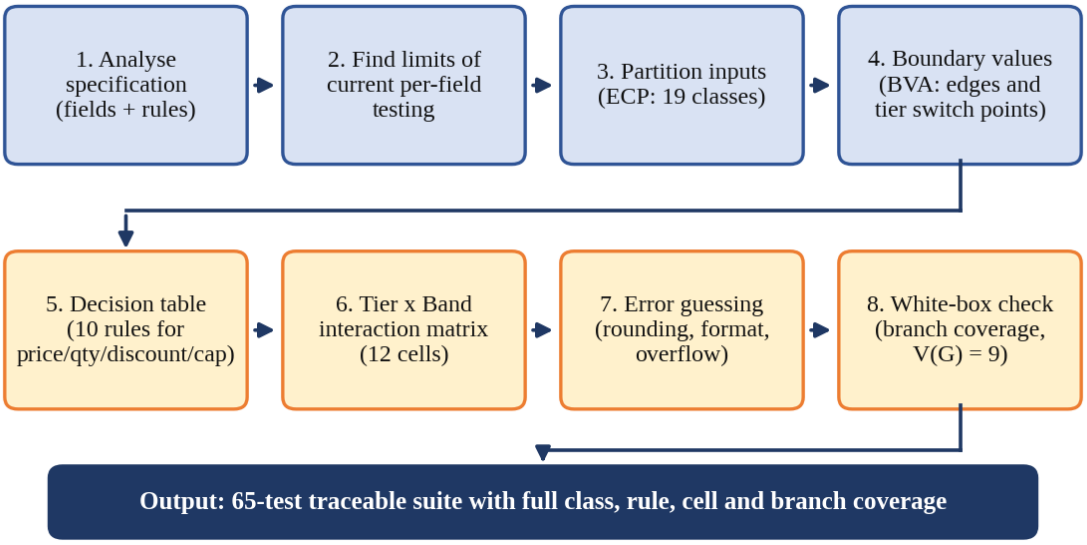


Figure 3: Workflow of the revised black-box test design strategy

#	Technique layer	Test IDs	Count	Defect type targeted
1	ECP, invalid classes (single fault)	TC-E01 to E11	11	Field validation, format and type errors
2	ECP, valid-class combinations (tier x band)	TC-M01 to M12	12	Interaction of valid classes under BR4
3	BVA, range edges	TC-B01 to B18	18	Off-by-one at minimum and maximum of each field
4	BVA, interaction boundaries	TC-C01 to C10	10	Tier switch points and cap edges
5	Decision table	TC-D01 to D10	10	Multi-fault logic and error reporting
6	Error guessing	TC-G01 to G04	4	Rounding, notation, overflow, empty submit
	Total		**65**	

4.2 Coverage of all input combinations

"All possible combinations" is achieved at the level of behaviour, and it is measurable. Each coverage item below is listed explicitly and mapped to the tests that cover it.

Coverage item	Items	Covered by	Coverage
Equivalence classes (price, qty, discount)	19	TC-M01 to M12 (8 valid), TC-E01 to E11 (11 invalid)	19 / 19 (100 %)
Valid class combinations (P x Q x D)	12	TC-M01 to M12	12 / 12 (100 %)
Decision-table rules	10	TC-D01 to D10	10 / 10 (100 %)
Range boundary values	18	TC-B01 to B18	18 / 18 (100 %)
Interaction boundary points (tier and cap edges)	10	TC-C01 to C10	10 / 10 (100 %)
Decision-node outcomes in the validation logic	16	Section 4.3	16 / 16 (100 %)

4.3 White-box cross-check of the method flow

Black-box tests are derived from the specification, so a white-box check confirms that they also exercise the control flow of the method that implements it. The validation and calculation logic is shown below in pseudocode, followed by its eight decision nodes.

```
errs = []
D1: if invalid(price):          errs += PriceError
D2: if invalid(qty):            errs += QtyError
D3: if invalid(discount):       errs += DiscountError
D4: if qty valid AND discount valid:
D5:    if qty < 10:   cap = 10
D6:    elif qty < 50: cap = 25   else: cap = 50
D7:    if discount > cap: errs += CapError
D8: if errs is empty: return round_half_up(price*qty*(1-discount/100), 2)
    else: reject(errs)
```

The control flow graph has 8 decision nodes, so the cyclomatic complexity is V(G) = 8 + 1 = **9**, meaning nine independent paths. The suite contains far more than nine tests and covers both outcomes of every node, as the table shows.

Node	Decision	TRUE outcome covered by	FALSE outcome covered by
D1	Price invalid?	TC-E01	TC-M01
D2	Quantity invalid?	TC-E05	TC-M01
D3	Discount invalid?	TC-E09	TC-M01
D4	Quantity and discount both valid?	TC-M01	TC-E05
D5	Quantity < 10?	TC-M01	TC-M05
D6	Quantity < 50?	TC-M05	TC-M09
D7	Discount > cap?	TC-M03	TC-M02
D8	Errors empty (accept)?	TC-M01	TC-E01

4.4 Execution and maintenance

The suite is small enough to run on every build as a parameterised, data-driven test, with the test data held in a table whose columns match Sections 3.1 to 3.5. Decision-table and matrix tests run first because they catch the highest-risk interaction defects. When a business rule changes, for example a new cap for quantity 50 and above, only the affected BR row, matrix cells and boundary tests need to be regenerated, because every test is traceable to a rule. If further fields such as coupon code or customer tier are added, all-pairs (pairwise) combination should be used so the suite grows slowly instead of multiplying.

5. Analysis, Evaluation and Justification

5.1 Seeded-defect analysis

To judge defect detection objectively, eleven plausible defects are seeded into the assumed implementation. Each is checked against the tests that would expose it. This is a logical (illustrative) analysis based on the assumed rules, not a measurement on real software.

ID	Seeded defect	Type	Found by current approach?	First layer that finds it	Detecting tests
M1	Price lower bound coded price > 1.00 (1.00 rejected)	Boundary	Yes	Per-field BVA	TC-B02
M2	Price upper bound coded price < 10000	Boundary	Yes	Per-field BVA	TC-B05
M3	Quantity upper bound coded qty < 100	Boundary	Yes	Per-field BVA	TC-B11

ID	Seeded defect	Type	Found by current approach?	First layer that finds it	Detecting tests
M4	Discount 0 treated as missing and rejected	Equivalence	Yes	Per-field ECP	TC-B14, M01
M5	Non-numeric discount gives NaN total, no error	Input class	Yes	Per-field ECP	TC-E11
M6	Price with three decimals accepted	Input class	Yes	Per-field ECP	TC-E04
M7	Tier 1 cap coded as 15 % instead of 10 %	Interaction	No	Tier x band matrix	TC-M03, C02, D02
M8	Tier 2 switch coded qty > 10 (qty 10 gets 10 % cap)	Interaction boundary	No	Cap boundaries	TC-C03, C04
M9	Tier 3 switch coded qty > 50 (qty 50 gets 25 % cap)	Interaction boundary	No	Cap boundaries	TC-C08, C09
M10	Validation stops at the first faulty field	Rule logic	No	Decision table	TC-D04, D07 to D10
M11	Floating-point rounding: 169.915 becomes 169.91	Computation	No	Error guessing	TC-G01

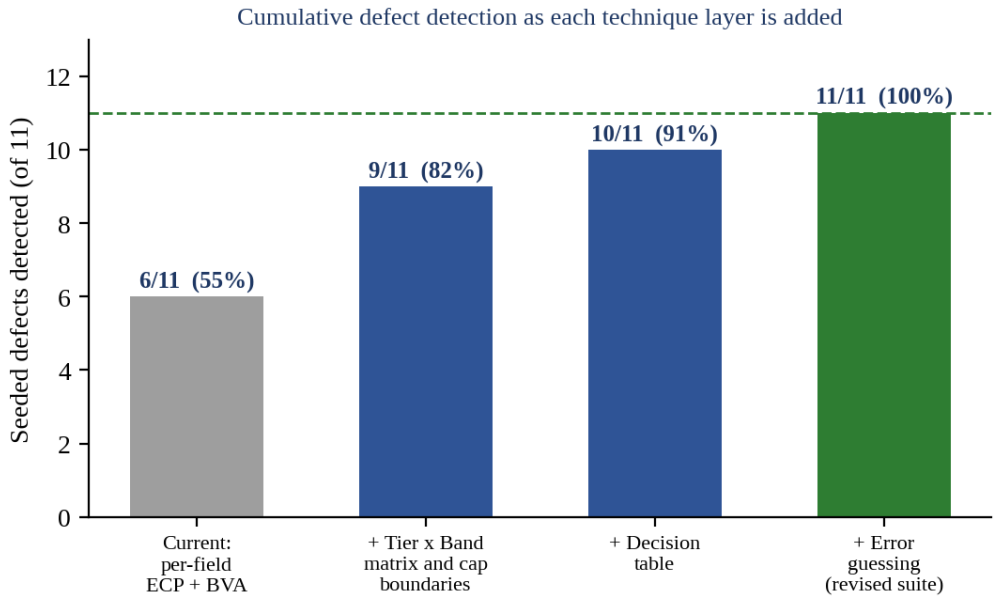


Figure 4: Cumulative seeded-defect detection as each technique layer is added

The current approach finds 6 of 11 defects (55 %), all of them field-level. Every defect it misses is an interaction, internal-boundary, multi-fault or computation defect, which matches the symptom reported in the scenario. The revised layers raise detection to 11 of 11.

5.2 Test completeness: current versus revised

Measure	Current (assumed)	Revised	Change
Equivalence classes recognised	14 of 19 (74 %)	19 of 19 (100 %)	+5 classes
Valid quantity x discount combinations exercised	2 of 12 (17 %)	12 of 12 (100 %)	+10 cells
Decision-table rules exercised	4 of 10 (40 %)	10 of 10 (100 %)	+6 rules
Interaction boundary points tested	0 of 10	10 of 10 (100 %)	+10 points
Decision-node outcomes (branch coverage)	14 of 16 (88 %)	16 of 16 (100 %)	+2 outcomes
Seeded defects detected	6 of 11 (55 %)	11 of 11 (100 %)	+5 defects

The "current" figures follow from the assumption in Section 2.1 (per-field tests with fixed nominal values for the other fields, with only the incidentally touched combinations counted). Note that branch coverage of the current approach already looks high at 88 %. The two outcomes it misses, quantity in tier 2 (D6 true) and discount above the cap (D7 true), are exactly where the escaped defects live. This is why a coverage percentage alone gives false confidence, and why coverage must be measured against rules and combinations.

5.3 Alternatives considered

Approach	Tests	Evaluation
Exhaustive testing	5,099,495,100	Complete in theory but impossible: about 162 years at one test per second.
Random testing	Unbounded	A random valid (quantity, discount) pair hits one specific combination such as (10, 11) with probability 1 in 5,100 (0.02 %), so boundary and cap defects are found only by luck.
Full cross-product of the 19 classes	245	Complete but wasteful: combining invalid classes with each other produces fault-masking tests (one error hides another) and repeats the same behaviour.
Per-field ECP and BVA (current)	Small	Cheap but only 55 % detection; blind to every interaction.
Revised layered suite	**65**	**100 % of classes, rules, cells and boundaries; 100 % seeded-defect detection; every test traceable to a business rule.**

The revised suite is about 3.8 times smaller than the full class cross-product (245 / 65) and roughly 78 million times smaller than exhaustive testing (5,099,495,100 / 65), while retaining complete coverage of every distinct behaviour.

5.4 Justification

Improved defect detection. Detection improves because each added technique targets a failure type the baseline cannot reach. The interaction matrix and cap-boundary tests expose conditional rules (M7 to M9). The decision table exposes rule-combination and error-reporting faults (M10). Error guessing exposes computation and format faults (M11). Single-fault invalid-class tests keep field-level errors from masking one another. Together they raise detection from 55 % to 100 % on the seeded set.

Improved test completeness. Completeness is defined against explicit, countable items (19 classes, 12 combinations, 10 rules, 28 boundary points, 16 decision outcomes) rather than against an unexplained number of tests. Each item maps to a business rule and to the test that covers it, so a reviewer can see exactly what is and is not tested.

Limitations and risks. The business rules and the defect set are assumed, so the figures illustrate the method rather than measure a real system. Black-box tests cannot prove the absence of hidden branches, which is why the white-box check in Section 4.3 is included. If the real specification differs, Sections 1.2 and 3.1 to 3.4 are re-derived from it using the same procedure.

6. Conclusion

The retail system's escaped defects came from interactions between price, quantity and discount, which a field-by-field black-box approach cannot detect. Equivalence partitioning refined by the discount-cap rule, boundary analysis extended to internal tier boundaries, decision table testing, a tier-by-band interaction matrix and targeted error guessing together form a 65-test suite. It covers every equivalence class, every combination of valid classes, every decision rule and every boundary, and a white-box check confirms that every branch of the validation logic is exercised. The result is higher defect detection (55 % to 100 % on the seeded set) and demonstrable completeness at a tiny fraction of the cost of exhaustive testing.

7. Mapping to Assessment Rubrics

Rubric criterion	Weight	Where addressed
Problem understanding and context analysis	25	Section 1 (scenario, assumed business rules, input-space size) and Section 2.1
Application of concepts	30	Section 3: ECP (3.1), BVA (3.2), decision table (3.3), interaction matrix and boundaries (3.4), error guessing (3.5)
Analytical and decision-making skills	20	Section 2 (limitations, worked example), Section 5 (seeded-defect analysis, alternatives considered)
Solution design and approach	15	Section 4 (layered strategy, coverage of all combinations, white-box cross-check, maintenance)

Rubric criterion	Weight	Where addressed
Clarity, justification and presentation	10	Section 5.4, Section 6, and the structured tables and Figures 1 to 4 throughout

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario	
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts	
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions	
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach	
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification	